

MODERNISER UN MONOLITHE ASP.NET MVC 4.8

SÉPARER PAR DOMAINE AVANT DE PARLER MICROSERVICES

CONTEXTE

SITUATION ACTUELLE

- Monolithe ASP.NET MVC .NET 4.8
- Code spaghetti ultra imbriqué
- Multiples projets et dépendances croisées
- Beaucoup d'interfaces... mal découpées
- Run + Change permanent

PROBLÈMES OBSERVÉS

- Couplage fort entre domaines
- Base de données partagée
- Dépendances circulaires
- Interfaces instables ou fourre-tout
- Effet domino lors des évolutions

**POURQUOI PAS DES
MICROSERVICES TOUT
DE SUITE ?**

RISQUE D'EXTRACTION IMMÉDIATE

- On distribue le spaghetti
- La base reste le point de couplage
- Complexité opérationnelle $\times 10$
- Debug plus difficile
- Incidents plus fréquents

MICROSERVICES $\neq$ SOLUTION MAGIQUE

Les microservices ajoutent :

- Réseau
- Résilience
- Versioning
- Observabilité
- DevOps avancé
- Gestion multi-dépôts

STRATÉGIE RÉALISTE

OBJECTIF

Rendre le monolithe **extractable**

Pas parfait. Pas académique. Mais structuré par domaines.

PRINCIPE

Moderniser en livrant.

Chaque évolution métier devient une opportunité de :

- Réduire le couplage
- Structurer par domaine
- Ajouter un filet de sécurité

SÉPARER PAR DOMAINE (DANS LE MONOLITHE)

ÉTAPE 1 – IDENTIFIER 4 À 8 DOMAINES

Méthode simple :

- Partir des features / écrans
- Lister les use cases majeurs
- Identifier les tables principales
- Repérer les dépendances externes

Livrable : 1 slide de cartographie métier.

ÉTAPE 2 – CRÉER UN MODULE PAR DOMAINE

Structure cible :

Modules/ Orders/ Billing/ Customers/

Objectif :

- Regrouper par responsabilité métier
- Déplacer progressivement le code
- Pas de refactor massif initial

ÉTAPE 3 – INTRODUIRE UNE FAÇADE DE DOMAINE

Exemple :

IOrdersModule

- CreateOrder()
- CancelOrder()

Règle :

Tout appel inter-domaine passe par la façade.

ÉTAPE 4 – SLICE VERTICAL À CHAQUE FEATURE

Pour chaque nouvelle demande :

- Déplacer le code touché dans le module
- Passer par la façade
- Ajouter 1 test filet
- Supprimer 1 dépendance toxique si possible

Budget : 10–20% de capacité par feature.

GÉRER LES INTERFACES EXISTANTES

CONSTAT

- Interfaces techniques partout
- IOrderService avec 25 méthodes
- Helpers globaux
- Interfaces qui masquent le couplage

CLASSIFICATION ACTIONNABLE

A — Façades métier

B — Ports techniques

C — Interfaces de confort

Objectif :



Remonter A



Encapsuler B



Déprécier C progressivement

NOUVELLE RÈGLE D'ÉQUIPE

- 1 façade par domaine
- Pas d'accès direct à la DB d'un autre domaine
- Pas de nouvelle dépendance transversale
- 1 test filet par feature

TIMELINE RÉALISTE

8 SEMAINES

S1 : Cartographie + domaine pilote

S2-4 : Structuration opportuniste

S5-8 : Domaine stabilisé

Mois 3 : Extraction possible

MATRICE DE DÉCISION : MICROSERVICES OU PAS ?

QUAND LES MICROSERVICES SONT PERTINENTS

- Domaines réellement autonomes
- Besoin de scalabilité différenciée
- Plusieurs équipes
- Déploiements indépendants requis
- Forte exigence d'isolation

QUAND ILS SONT PRÉMATURÉS

- Frontières floues
- Base partagée fortement couplée
- CI/CD immature
- Observabilité faible
- Équipe unique

IMPACT : DÉVELOPPEMENT & SOURCE

	Monolithe	Modulaire	Microservices
Dev	Simple, effet domino	Refactor localisé	Autonomie, complexité distribuée
Source	1 repo, collisions	Ownership par module	Repo par service, fragmentation

IMPACT : CI/CD & BASE DE DONNÉES

	Monolithe	Modulaire	Microservices
CI/CD	Pipeline unique, déploiement risqué	Pipeline fiable, artefact global	Déploiement indépendant, coût DevOps élevé
DB 	Transactions simples, couplage maximal	BD encore partagée	DB par service, cohérence éventuelle, reporting distribué

RECOMMANDATION

TRAJECTOIRE PRAGMATIQUE

1. Monolithe modulaire par domaines
2. Façades stables
3. Réduction du couplage
4. Tests filet
5. Extraction ciblée quand prêt

MESSAGE CLÉ FINAL

On ne migre pas vers des microservices.

On rend le monolithe extractable.

Les microservices sont une conséquence, pas un point de départ.

QUESTIONS